第5章

基础构建模块

第4章介绍了构造线程安全类时采用的一些技术，例如将线程安全性委托给现有的线程安全类。委托是创建线程安全类的一个最有效的策略：只需让现有的线程安全类管理所有的状态即可。

Java平台类库包含了丰富的并发基础构建模块，例如线程安全的容器类以及各种用于协调多个相互协作的线程控制流的同步工具类（Synchronizer）。本章将介绍其中一些最有用的并发构建模块，特别是在Java 5.0和Java 6中引入的一些新模块，以及在使用这些模块来构造并发应用程序时的一些常用模式。

5.1 同步容器类

同步容器类包括Vector和Hashtable，二者是早期JDK的一部分，此外还包括在JDK 1.2中添加的一些功能相似的类，这些同步的封装器类是由Collections.synchronizedXxx等工厂方法创建的。这些类实现线程安全的方式是：将它们的状态封装起来，并对每个公有方法都进行同步，使得每次只有一个线程能访问容器的状态。

5.1.1 同步容器类的问题

同步容器类都是线程安全的，但在某些情况下可能需要额外的客户端加锁来保护复合操作。容器上常见的复合操作包括：迭代（反复访问元素，直到遍历完容器中所有元素）、跳转（根据指定顺序找到当前元素的下一个元素）以及条件运算，例如“若没有则添加”（检查在Map中是否存在键值K，如果没有，就加入二元组(K,V)）。在同步容器类中，这些复合操作在没有客户端加锁的情况下仍然是线程安全的，但当其他线程并发地修改容器时，它们可能会表现出意料之外的行为。

程序清单5-1给出了在Vector中定义的两个方法：getLast和deleteLast，它们都会执行“先检查再运行”操作。每个方法首先都获得数组的大小，然后通过结果来获取或删除最后一个元素。

程序清单5-1 Vector上可能导致混乱结果的复合操作

```
public static Object getLast(Vector list) {
    int lastIndex = list.size() - 1;
    return list.get(lastIndex);
}
```

```
public static void deleteLast(Vector list) {
    int lastIndex = list.size() - 1;
    list.remove(lastIndex);
}
```

这些方法看似没有任何问题，从某种程度上来看也确实如此——无论多少个线程同时调用它们，也不破坏 Vector。但从这些方法的调用者角度来看，情况就不同了。如果线程 A 在包含 10 个元素的 Vector 上调用 getLast，同时线程 B 在同一个 Vector 上调用 deleteLast，这些操作的交替执行如图 5-1 所示，getLast 将抛出 ArrayIndexOutOfBoundsException 异常。在调用 size 与调用 getLast 这两个操作之间，Vector 变小了，因此在调用 size 时得到的索引值将不再有效。这种情况很好地遵循了 Vector 的规范——如果请求一个不存在的元素，那么将抛出一个异常。但这并不是 getLast 的调用者所希望得到的结果（即使在并发修改的情况下也不希望看到），除非 Vector 从一开始就是空的。

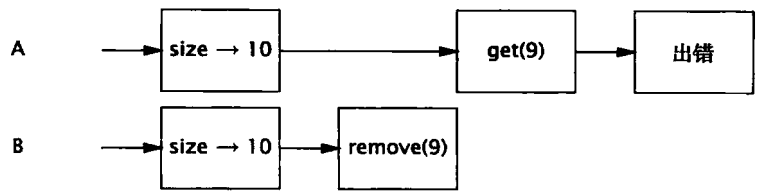

图 5-1　交替调用 getLast 和 deleteLast 时将抛出 ArrayIndexOutOfBoundsException

由于同步容器类要遵守同步策略，即支持客户端加锁[⊖]，因此可能会创建一些新的操作，只要我们知道应该使用哪一个锁，那么这些新操作就与容器的其他操作一样都是原子操作。同步容器类通过其自身的锁来保护它的每个方法。通过获得容器类的锁，我们可以使 getLast 和 deleteLast 成为原子操作，并确保 Vector 的大小在调用 size 和 get 之间不会发生变化，如程序清单 5-2 所示。

程序清单 5-2　在使用客户端加锁的 Vector 上的复合操作

```
public static Object getLast(Vector list) {
    synchronized (list) {
        int lastIndex = list.size() - 1;
        return list.get(lastIndex);
    }
}

public static void deleteLast(Vector list) {
    synchronized (list) {
        int lastIndex = list.size() - 1;
        list.remove(lastIndex);
    }
}
```

⊖ 这只在 Java 5.0 的 Javadoc 中作为迭代示例简要地提了一下。

在调用 size 和相应的 get 之间，Vector 的长度可能会发生变化，这种风险在对 Vector 中的元素进行迭代时仍然会出现，如程序清单 5-3 所示。

程序清单 5-3 可能抛出 ArrayIndexOutOfBoundsException 的迭代操作

```
for (int i = 0; i < vector.size(); i++)
    doSomething(vector.get(i));
```

这种迭代操作的正确性要依赖于运气，即在调用 size 和 get 之间没有线程会修改 Vector。在单线程环境中，这种假设完全成立，但在有其他线程并发地修改 Vector 时，则可能导致麻烦。与 getLast 一样，如果在对 Vector 进行迭代时，另一个线程删除了一个元素，并且这两个操作交替执行，那么这种迭代方法将抛出 ArrayIndexOutOfBoundsException 异常。

虽然在程序清单 5-3 的迭代操作中可能抛出异常，但并不意味着 Vector 就不是线程安全的。Vector 的状态仍然是有效的，而抛出的异常也与其规范保持一致。然而，像在读取最后一个元素或者迭代等这样的简单操作中抛出异常显然不是人们所期望的。

我们可以通过在客户端加锁来解决不可靠迭代的问题，但要牺牲一些伸缩性。通过在迭代期间持有 Vector 的锁，可以防止其他线程在迭代期间修改 Vector，如程序清单 5-4 所示。然而，这同样会导致其他线程在迭代期间无法访问它，因此降低了并发性。

程序清单 5-4 带有客户端加锁的迭代

```
synchronized (vector) {
    for (int i = 0; i < vector.size(); i++)
        doSomething(vector.get(i));
}
```

5.1.2 迭代器与 ConcurrentModificationException

为了将问题阐述清楚，我们使用了 Vector，虽然这是一个“古老”的容器类。然而，许多“现代”的容器类也并没有消除复合操作中的问题。无论在直接迭代还是在 Java 5.0 引入的 for-each 循环语法中，对容器类进行迭代的标准方式都是使用 Iterator。然而，如果有其他线程并发地修改容器，那么即使是使用迭代器也无法避免在迭代期间对容器加锁。在设计同步容器类的迭代器时并没有考虑到并发修改的问题，并且它们表现出的行为是“及时失败”（fail-fast）的。这意味着，当它们发现容器在迭代过程中被修改时，就会抛出一个 ConcurrentModificationException 异常。

这种“及时失败”的迭代器并不是一种完备的处理机制，而只是“善意地”捕获并发错误，因此只能作为并发问题的预警指示器。它们采用的实现方式是，将计数器的变化与容器关联起来：如果在迭代期间计数器被修改，那么 hasNext 或 next 将抛出 ConcurrentModificationException。然而，这种检查是在没有同步的情况下进行的，因此可能会看到失效的计数值，而迭代器可能并没有意识到已经发生了修改。这是一种设计上的权

衡，从而降低并发修改操作的检测代码[⊖]对程序性能带来的影响。

程序清单 5-5 说明了如何使用 for-each 循环语法对 List 容器进行迭代。从内部来看，javac 将生成使用 Iterator 的代码，反复调用 hasNext 和 next 来迭代 List 对象。与迭代 Vector 一样，要想避免出现 ConcurrentModificationException，就必须在迭代过程持有容器的锁。

程序清单 5-5 通过 Iterator 来迭代 List

```
List<Widget> widgetList
    = Collections.synchronizedList(new ArrayList<Widget>());
...
// 可能抛出 ConcurrentModificationException
for (Widget w : widgetList)
    doSomething(w);
```

然而，有时候开发人员并不希望在迭代期间对容器加锁。例如，某些线程在可以访问容器之前，必须等待迭代过程结束，如果容器的规模很大，或者在每个元素上执行操作的时间很长，那么这些线程将长时间等待。同样，如果容器像程序清单 5-4 中那样加锁，那么在调用 doSomething 时将持有一个锁，这可能会产生死锁（请参见第 10 章）。即使不存在饥饿或者死锁等风险，长时间地对容器加锁也会降低程序的可伸缩性。持有锁的时间越长，那么在锁上的竞争就可能越激烈，如果许多线程都在等待锁被释放，那么将极大地降低吞吐量和 CPU 的利用率（请参见第 11 章）。

如果不希望在迭代期间对容器加锁，那么一种替代方法就是“克隆”容器，并在副本上进行迭代。由于副本被封闭在线程内，因此其他线程不会在迭代期间对其进行修改，这样就避免了抛出 ConcurrentModificationException（在克隆过程中仍然需要对容器加锁）。在克隆容器时存在显著的性能开销。这种方式的好坏取决于多个因素，包括容器的大小，在每个元素上执行的工作，迭代操作相对于容器其他操作的调用频率，以及在响应时间和吞吐量等方面的需求。

5.1.3 隐藏迭代器

虽然加锁可以防止迭代器抛出 ConcurrentModificationException，但你必须要记住在所有对共享容器进行迭代的地方都需要加锁。实际情况要更加复杂，因为在某些情况下，迭代器会隐藏起来，如程序清单 5-6 中的 HiddenIterator 所示。在 HiddenIterator 中没有显式的迭代操作，但在粗体标出的代码中将执行迭代操作。编译器将字符串的连接操作转换为调用 StringBuilder.append(Object)，而这个方法又会调用容器的 toString 方法，标准容器的 toString 方法将迭代容器，并在每个元素上调用 toString 来生成容器内容的格式化表示。

程序清单 5-6 隐藏在字符串连接中的迭代操作（不要这么做）

```
public class HiddenIterator {
    @GuardedBy("this")
```

⊖ 在单线程代码中也可能抛出 ConcurrentModificationException 异常。当对象直接从容器中删除而不是通过 Iterator.remove 来删除时，就会抛出这个异常。

```
    private final Set<Integer> set = new HashSet<Integer>();

    public synchronized void add(Integer i) { set.add(i); }
    public synchronized void remove(Integer i) { set.remove(i); }

    public void addTenThings() {
        Random r = new Random();
        for (int i = 0; i < 10; i++)
            add(r.nextInt());
        System.out.println("DEBUG: added ten elements to " + set);
    }
}
```

addTenThings 方法可能会抛出 ConcurrentModificationException，因为在生成调试消息的过程中，toString 对容器进行迭代。当然，真正的问题在于 HiddenIteracor 不是线程安全的。在使用 println 中的 set 之前必须首先获取 HiddenIterator 的锁，但在调试代码和日志代码中通常会忽视这个要求。

这里得到的教训是，如果状态与保护它的同步代码之间相隔越远，那么开发人员就越容易忘记在访问状态时使用正确的同步。如果 HiddenIterator 用 synchronizedSet 来包装 HashSet，并且对同步代码进行封装，那么就不会发生这种错误。

> 正如封装对象的状态有助于维持不变性条件一样，封装对象的同步机制同样有助于确保实施同步策略。

容器的 hashCode 和 equals 等方法也会间接地执行迭代操作，当容器作为另一个容器的元素或键值时，就会出现这种情况。同样，containsAll、removeAll 和 retainAll 等方法，以及把容器作为参数的构造函数，都会对容器进行迭代。所有这些间接的迭代操作都可能抛出 ConcurrentModificationException。

5.2 并发容器

Java 5.0 提供了多种并发容器类来改进同步容器的性能。同步容器将所有对容器状态的访问都串行化，以实现它们的线程安全性。这种方法的代价是严重降低并发性，当多个线程竞争容器的锁时，吞吐量将严重减低。

另一方面，并发容器是针对多个线程并发访问设计的。在 Java 5.0 中增加了 ConcurrentHashMap，用来替代同步且基于散列的 Map，以及 CopyOnWriteArrayList，用于在遍历操作为主要操作的情况下代替同步的 List。在新的 ConcurrentMap 接口中增加了对一些常见复合操作的支持，例如“若没有则添加”、替换以及有条件删除等。

> 通过并发容器来代替同步容器，可以极大地提高伸缩性并降低风险。

Java 5.0 增加了两种新的容器类型：Queue 和 BlockingQueue。Queue 用来临时保存一组等

待处理的元素。它提供了几种实现，包括：ConcurrentLinkedQueue，这是一个传统的先进先出队列，以及 PriorityQueue，这是一个（非并发的）优先队列。Queue 上的操作不会阻塞，如果队列为空，那么获取元素的操作将返回空值。虽然可以用 List 来模拟 Queue 的行为——事实上，正是通过 LinkedList 来实现 Queue 的，但还需要一个 Queue 的类，因为它能去掉 List 的随机访问需求，从而实现更高效的并发。

BlockingQueue 扩展了 Queue，增加了可阻塞的插入和获取等操作。如果队列为空，那么获取元素的操作将一直阻塞，直到队列中出现一个可用的元素。如果队列已满（对于有界队列来说），那么插入元素的操作将一直阻塞，直到队列中出现可用的空间。在“生产者－消费者”这种设计模式中，阻塞队列是非常有用的，5.3 节将会详细介绍。

正如 ConcurrentHashMap 用于代替基于散列的同步 Map，Java 6 也引入了 ConcurrentSkipListMap 和 ConcurrentSkipListSet，分别作为同步的 SortedMap 和 SortedSet 的并发替代品（例如用 synchronizedMap 包装的 TreeMap 或 TreeSet）。

5.2.1 ConcurrentHashMap

同步容器类在执行每个操作期间都持有一个锁。在一些操作中，例如 HashMap.get 或 List.contains，可能包含大量的工作：当遍历散列桶或链表来查找某个特定的对象时，必须在许多元素上调用 equals（而 equals 本身还包含一定的计算量）。在基于散列的容器中，如果 hashCode 不能很均匀地分布散列值，那么容器中的元素就不会均匀地分布在整个容器中。某些情况下，某个糟糕的散列函数还会把一个散列表变成线性链表。当遍历很长的链表并且在某些或者全部元素上调用 equals 方法时，会花费很长的时间，而其他线程在这段时间内都不能访问该容器。

与 HashMap 一样，ConcurrentHashMap 也是一个基于散列的 Map，但它使用了一种完全不同的加锁策略来提供更高的并发性和伸缩性。ConcurrentHashMap 并不是将每个方法都在同一个锁上同步并使得每次只能有一个线程访问容器，而是使用一种粒度更细的加锁机制来实现更大程度的共享，这种机制称为分段锁（Lock Striping，请参见 11.4.3 节）。在这种机制中，任意数量的读取线程可以并发地访问 Map，执行读取操作的线程和执行写入操作的线程可以并发地访问 Map，并且一定数量的写入线程可以并发地修改 Map。ConcurrentHashMap 带来的结果是，在并发访问环境下将实现更高的吞吐量，而在单线程环境中只损失非常小的性能。

ConcurrentHashMap 与其他并发容器一起增强了同步容器类：它们提供的迭代器不会抛出 ConcurrentModificationException，因此不需要在迭代过程中对容器加锁。ConcurrentHashMap 返回的迭代器具有弱一致性（Weakly Consistent），而并非“及时失败”。弱一致性的迭代器可以容忍并发的修改，当创建迭代器时会遍历已有的元素，并可以（但是不保证）在迭代器被构造后将修改操作反映给容器。

尽管有这些改进，但仍然有一些需要权衡的因素。对于一些需要在整个 Map 上进行计算的方法，例如 size 和 isEmpty，这些方法的语义被略微减弱了以反映容器的并发特性。由于 size 返回的结果在计算时可能已经过期了，它实际上只是一个估计值，因此允许 size 返回一个近似值而不是一个精确值。虽然这看上去有些令人不安，但事实上 size 和 isEmpty 这样的方法在并

发环境下的用处很小，因为它们的返回值总在不断变化。因此，这些操作的需求被弱化了，以换取对其他更重要操作的性能优化，包括 get、put、containsKey 和 remove 等。

在 ConcurrentHashMap 中没有实现对 Map 加锁以提供独占访问。在 Hashtable 和 synchronizedMap 中，获得 Map 的锁能防止其他线程访问这个 Map。在一些不常见的情况中需要这种功能，例如通过原子方式添加一些映射，或者对 Map 迭代若干次并在此期间保持元素顺序相同。然而，总体来说这种权衡还是合理的，因为并发容器的内容会持续变化。

与 Hashtable 和 synchronizedMap 相比，ConcurrentHashMap 有着更多的优势以及更少的劣势，因此在大多数情况下，用 ConcurrentHashMap 来代替同步 Map 能进一步提高代码的可伸缩性。只有当应用程序需要加锁 Map 以进行独占访问[⊖]时，才应该放弃使用 ConcurrentHashMap。

5.2.2 额外的原子 Map 操作

由于 ConcurrentHashMap 不能被加锁来执行独占访问，因此我们无法使用客户端加锁来创建新的原子操作，例如 4.4.1 节中对 Vector 增加原子操作“若没有则添加”。但是，一些常见的复合操作，例如“若没有则添加”、“若相等则移除（Remove-If-Equal)”和“若相等则替换（Replace-If-Equal)”等，都已经实现为原子操作并且在 ConcurrentMap 的接口中声明，如程序清单 5-7 所示。如果你需要在现有的同步 Map 中添加这样的功能，那么很可能就意味着应该考虑使用 ConcurrentMap 了。

程序清单 5-7 ConcurrentMap 接口

```
public interface ConcurrentMap<K,V> extends Map<K,V> {
//仅当 K 没有相应的映射值时才插入
    V putIfAbsent(K key, V value);

//仅当 K 被映射到 V 时才移除
    boolean remove(K key, V value);

//仅当 K 被映射到 oldValue 时才替换为 newValue
    boolean replace(K key, V oldValue, V newValue);

//仅当 K 被映射到某个值时才替换为 newValue
    V replace(K key, V newValue);
}
```

5.2.3 CopyOnWriteArrayList

CopyOnWriteArrayList 用于替代同步 List，在某些情况下它提供了更好的并发性能，并且在迭代期间不需要对容器进行加锁或复制。（类似地，CopyOnWriteArraySet 的作用是替代同步 Set。）

“写入时复制 (Copy-On-Write)”容器的线程安全性在于，只要正确地发布一个事实不可

⊖ 或者需要依赖于同步 Map 带来的一些其他作用。

变的对象，那么在访问该对象时就不再需要进一步的同步。在每次修改时，都会创建并重新发布一个新的容器副本，从而实现可变性。“写入时复制”容器的迭代器保留一个指向底层基础数组的引用，这个数组当前位于迭代器的起始位置，由于它不会被修改，因此在对其进行同步时只需确保数组内容的可见性。因此，多个线程可以同时对这个容器进行迭代，而不会彼此干扰或者与修改容器的线程相互干扰。“写入时复制”容器返回的迭代器不会抛出 ConcurrentModificationException，并且返回的元素与迭代器创建时的元素完全一致，而不必考虑之后修改操作所带来的影响。

显然，每当修改容器时都会复制底层数组，这需要一定的开销，特别是当容器的规模较大时。仅当迭代操作远远多于修改操作时，才应该使用“写入时复制”容器。这个准则很好地描述了许多事件通知系统：在分发通知时需要迭代已注册监听器链表，并调用每一个监听器，在大多数情况下，注册和注销事件监听器的操作远少于接收事件通知的操作。（关于“写入时复制”的更多信息请参见 [CPJ 2.4.4]。）

5.3 阻塞队列和生产者－消费者模式

阻塞队列提供了可阻塞的 put 和 take 方法，以及支持定时的 offer 和 poll 方法。如果队列已经满了，那么 put 方法将阻塞直到有空间可用；如果队列为空，那么 take 方法将会阻塞直到有元素可用。队列可以是有界的也可以是无界的，无界队列永远都不会充满，因此无界队列上的 put 方法也永远不会阻塞。

阻塞队列支持生产者－消费者这种设计模式。该模式将“找出需要完成的工作”与“执行工作”这两个过程分离开来，并把工作项放入一个“待完成”列表中以便在随后处理，而不是找出后立即处理。生产者－消费者模式能简化开发过程，因为它消除了生产者类和消费者类之间的代码依赖性，此外，该模式还将生产数据的过程与使用数据的过程解耦开来以简化工作负载的管理，因为这两个过程在处理数据的速率上有所不同。

在基于阻塞队列构建的生产者－消费者设计中，当数据生成时，生产者把数据放入队列，而当消费者准备处理数据时，将从队列中获取数据。生产者不需要知道消费者的标识或数量，或者它们是否是唯一的生产者，而只需将数据放入队列即可。同样，消费者也不需要知道生产者是谁，或者工作来自何处。BlockingQueue 简化了生产者－消费者设计的实现过程，它支持任意数量的生产者和消费者。一种最常见的生产者－消费者设计模式就是线程池与工作队列的组合，在 Executor 任务执行框架中就体现了这种模式，这也是第 6 章和第 8 章的主题。

以两个人洗盘子为例，二者的劳动分工也是一种生产者－消费者模式：其中一个人把洗好的盘子放在盘架上，而另一个人从盘架上取出盘子并把它们烘干。在这个示例中，盘架相当于阻塞队列。如果盘架上没有盘子，那么消费者会一直等待，直到有盘子需要烘干。如果盘架放满了，那么生产者会停止清洗直到盘架上有更多的空间。我们可以将这种类比扩展为多个生产者（虽然可能存在对水槽的竞争）和多个消费者，每个工人只需与盘架打交道。人们不需要知道究竟有多少生产者或消费者，或者谁生产了某个指定的工作项。

“生产者”和“消费者”的角色是相对的，某种环境中的消费者在另一种不同的环境中可能会成为生产者。烘干盘子的工人将“消费”洗干净的湿盘子，而产生烘干的盘子。第三个人

把洗干净的盘子整理好，在这种情况中，烘干盘子的工人既是消费者，也是生产者，从而就有了两个共享的工作队列（每个队列都可能阻塞烘干工作的运行）。

阻塞队列简化了消费者程序的编码，因为 take 操作会一直阻塞直到有可用的数据。如果生产者不能尽快地产生工作项使消费者保持忙碌，那么消费者就只能一直等待，直到有工作可做。在某些情况下，这种方式是非常合适的（例如，在服务器应用程序中，没有任何客户请求服务），而在其他一些情况下，这也表示需要调整生产者线程数量和消费者线程数量之间的比率，从而实现更高的资源利用率（例如，在“网页爬虫 [Web Crawler]”或其他应用程序中，有无穷的工作需要完成）。

如果生产者生成工作的速率比消费者处理工作的速率快，那么工作项会在队列中累积起来，最终耗尽内存。同样，put 方法的阻塞特性也极大地简化了生产者的编码。如果使用有界队列，那么当队列充满时，生产者将阻塞并且不能继续生成工作，而消费者就有时间来赶上工作处理进度。

阻塞队列同样提供了一个 offer 方法，如果数据项不能被添加到队列中，那么将返回一个失败状态。这样你就能够创建更多灵活的策略来处理负荷过载的情况，例如减轻负载，将多余的工作项序列化并写入磁盘，减少生产者线程的数量，或者通过某种方式来抑制生产者线程。

在构建高可靠的应用程序时，有界队列是一种强大的资源管理工具：它们能抑制并防止产生过多的工作项，使应用程序在负荷过载的情况下变得更加健壮。

虽然生产者－消费者模式能够将生产者和消费者的代码彼此解耦开来，但它们的行为仍然会通过共享工作队列间接地耦合在一起。开发人员总会假设消费者处理工作的速率能赶上生产者生成工作项的速率，因此通常不会为工作队列的大小设置边界，但这将导致在之后需要重新设计系统架构。因此，应该尽早地通过阻塞队列在设计中构建资源管理机制——这件事情做得越早，就越容易。在许多情况下，阻塞队列能使这项工作更加简单，如果阻塞队列并不完全符合设计需求，那么还可以通过信号量 (Semaphore) 来创建其他的阻塞数据结构（请参见 5.5.3 节）。

在类库中包含了 BlockingQueue 的多种实现，其中，LinkedBlockingQueue 和 ArrayBlockingQueue 是 FIFO 队列，二者分别与 LinkedList 和 ArrayList 类似，但比同步 List 拥有更好的并发性能。PriorityBlockingQueue 是一个按优先级排序的队列，当你希望按照某种顺序而不是 FIFO 来处理元素时，这个队列将非常有用。正如其他有序的容器一样，PriorityBlockingQueue 既可以根据元素的自然顺序来比较元素（如果它们实现了 Comparable 方法），也可以使用 Comparator 来比较。

最后一个 BlockingQueue 实现是 SynchronousQueue，实际上它不是一个真正的队列，因为它不会为队列中元素维护存储空间。与其他队列不同的是，它维护一组线程，这些线程在等待着把元素加入或移出队列。如果以洗盘子的比喻为例，那么这就相当于没有盘架，而是将洗好的盘子直接放入下一个空闲的烘干机中。这种实现队列的方式看似很奇怪，但由于可以直接交付工作，从而降低了将数据从生产者移动到消费者的延迟。（在传统的队列中，在一个工作单元可以交付之前，必须通过串行方式首先完成入列 [Enqueue] 或者出列 [Dequeue] 等操作。）直

接交付方式还会将更多关于任务状态的信息反馈给生产者。当交付被接受时，它就知道消费者已经得到了任务，而不是简单地把任务放入一个队列——这种区别就好比将文件直接交给同事，还是将文件放到她的邮箱中并希望她能尽快拿到文件。因为 SynchronousQueue 没有存储功能，因此 put 和 take 会一直阻塞，直到有另一个线程已经准备好参与到交付过程中。仅当有足够多的消费者，并且总是有一个消费者准备好获取交付的工作时，才适合使用同步队列。

5.3.1　示例：桌面搜索

有一种类型的程序适合被分解为生产者和消费者，例如代理程序，它将扫描本地驱动器上的文件并建立索引以便随后进行搜索，类似于某些桌面搜索程序或者 Windows 索引服务。在程序清单 5-8 的 DiskCrawler 中给出了一个生产者任务，即在某个文件层次结构中搜索符合索引标准的文件，并将它们的名称放入工作队列。而且，在 Indexer 中还给出了一个消费者任务，即从队列中取出文件名称并对它们建立索引。

程序清单 5-8　桌面搜索应用程序中的生产者任务和消费者任务

```
public class FileCrawler implements Runnable {
    private final BlockingQueue<File> fileQueue;
    private final FileFilter fileFilter;
    private final File root;
    ...
    public void run() {
        try {
            crawl(root);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void crawl(File root) throws InterruptedException {
        File[] entries = root.listFiles(fileFilter);
        if (entries != null) {
            for (File entry : entries)
                if (entry.isDirectory())
                    crawl(entry);
                else if (!alreadyIndexed(entry))
                    fileQueue.put(entry);
        }
    }
}

public class Indexer implements Runnable {
    private final BlockingQueue<File> queue;

    public Indexer(BlockingQueue<File> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
            while (true)
```

```
                indexFile(queue.take());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
```

生产者－消费者模式提供了一种适合线程的方法将桌面搜索问题分解为更简单的组件。将文件遍历与建立索引等功能分解为独立的操作，比将所有功能都放到一个操作中实现有着更高的代码可读性和可重用性：每个操作只需完成一个任务，并且阻塞队列将负责所有的控制流，因此每个功能的代码都更加简单和清晰。

生产者－消费者模式同样能带来许多性能优势。生产者和消费者可以并发地执行。如果一个是 I/O 密集型，另一个是 CPU 密集型，那么并发执行的吞吐率要高于串行执行的吞吐率。如果生产者和消费者的并行度不同，那么将它们紧密耦合在一起会把整体并行度降低为二者中更小的并行度。

在程序清单 5-9 中启动了多个爬虫程序和索引建立程序，每个程序都在各自的线程中运行。前面曾讲，消费者线程永远不会退出，因而程序无法终止，第 7 章将介绍多种技术来解决这个问题。虽然这个示例使用了显式管理的线程，但许多生产者－消费者设计也可以通过 Executor 任务执行框架来实现，其本身也使用了生产者－消费者模式。

程序清单 5-9 启动桌面搜索

```
public static void startIndexing(File[] roots) {
    BlockingQueue<File> queue = new LinkedBlockingQueue<File>(BOUND);
    FileFilter filter = new FileFilter() {
        public boolean accept(File file) { return true; }
    };

    for (File root : roots)
        new Thread(new FileCrawler(queue, filter, root)).start();

    for (int i = 0; i < N_CONSUMERS; i++)
        new Thread(new Indexer(queue)).start();
}
```

5.3.2 串行线程封闭

在 java.util.concurrent 中实现的各种阻塞队列都包含了足够的内部同步机制，从而安全地将对象从生产者线程发布到消费者线程。

对于可变对象，生产者－消费者这种设计与阻塞队列一起，促进了串行线程封闭，从而将对象所有权从生产者交付给消费者。线程封闭对象只能由单个线程拥有，但可以通过安全地发布该对象来“转移”所有权。在转移所有权后，也只有另一个线程能获得这个对象的访问权限，并且发布对象的线程不会再访问它。这种安全的发布确保了对象状态对于新的所有者来说是可见的，并且由于最初的所有者不会再访问它，因此对象将被封闭在新的线程中。新的所有

者线程可以对该对象做任意修改，因为它具有独占的访问权。

对象池利用了串行线程封闭，将对象“借给”一个请求线程。只要对象池包含足够的内部同步来安全地发布池中的对象，并且只要客户代码本身不会发布池中的对象，或者在将对象返回给对象池后就不再使用它，那么就可以安全地在线程之间传递所有权。

我们也可以使用其他发布机制来传递可变对象的所有权，但必须确保只有一个线程能接受被转移的对象。阻塞队列简化了这项工作。除此之外，还可以通过 ConcurrentMap 的原子方法 remove 或者 AtomicReference 的原子方法 compareAndSet 来完成这项工作。

5.3.3 双端队列与工作密取

Java 6 增加了两种容器类型，Deque（发音为“deck”）和 BlockingDeque，它们分别对 Queue 和 BlockingQueue 进行了扩展。Deque 是一个双端队列，实现了在队列头和队列尾的高效插入和移除。具体实现包括 ArrayDeque 和 LinkedBlockingDeque。

正如阻塞队列适用于生产者－消费者模式，双端队列同样适用于另一种相关模式，即工作密取（Work Stealing）。在生产者－消费者设计中，所有消费者有一个共享的工作队列，而在工作密取设计中，每个消费者都有各自的双端队列。如果一个消费者完成了自己双端队列中的全部工作，那么它可以从其他消费者双端队列末尾秘密地获取工作。密取工作模式比传统的生产者－消费者模式具有更高的可伸缩性，这是因为工作者线程不会在单个共享的任务队列上发生竞争。在大多数时候，它们都只是访问自己的双端队列，从而极大地减少了竞争。当工作者线程需要访问另一个队列时，它会从队列的尾部而不是从头部获取工作，因此进一步降低了队列上的竞争程度。

工作密取非常适用于既是消费者也是生产者问题——当执行某个工作时可能导致出现更多的工作。例如，在网页爬虫程序中处理一个页面时，通常会发现有更多的页面需要处理。类似的还有许多搜索图的算法，例如在垃圾回收阶段对堆进行标记，都可以通过工作密取机制来实现高效并行。当一个工作线程找到新的任务单元时，它会将其放到自己队列的末尾（或者在工作共享设计模式中，放入其他工作者线程的队列中）。当双端队列为空时，它会在另一个线程的队列队尾查找新的任务，从而确保每个线程都保持忙碌状态。

5.4 阻塞方法与中断方法

线程可能会阻塞或暂停执行，原因有多种：等待 I/O 操作结束，等待获得一个锁，等待从 Thread.sleep 方法中醒来，或是等待另一个线程的计算结果。当线程阻塞时，它通常被挂起，并处于某种阻塞状态（BLOCKED、WAITING 或 TIMED_WAITING）。阻塞操作与执行时间很长的普通操作的差别在于，被阻塞的线程必须等待某个不受它控制的事件发生后才能继续执行，例如等待 I/O 操作完成，等待某个锁变成可用，或者等待外部计算的结束。当某个外部事件发生时，线程被置回 RUNNABLE 状态，并可以再次被调度执行。

BlockingQueue 的 put 和 take 等方法会抛出受检查异常（Checked Exception）InterruptedException，这与类库中其他一些方法的做法相同，例如 Thread.sleep。当某方法抛出 Interrupted-

Exception 时，表示该方法是一个阻塞方法，如果这个方法被中断，那么它将努力提前结束阻塞状态。

Thread 提供了 interrupt 方法，用于中断线程或者查询线程是否已经被中断。每个线程都有一个布尔类型的属性，表示线程的中断状态，当中断线程时将设置这个状态。

中断是一种协作机制。一个线程不能强制其他线程停止正在执行的操作而去执行其他的操作。当线程 A 中断 B 时，A 仅仅是要求 B 在执行到某个可以暂停的地方停止正在执行的操作——前提是如果线程 B 愿意停止下来。虽然在 API 或者语言规范中并没有为中断定义任何特定应用级别的语义，但最常使用中断的情况就是取消某个操作。方法对中断请求的响应度越高，就越容易及时取消那些执行时间很长的操作。

当在代码中调用了一个将抛出 InterruptedException 异常的方法时，你自己的方法也就变成了一个阻塞方法，并且必须要处理对中断的响应。对于库代码来说，有两种基本选择：

传递 InterruptedException。避开这个异常通常是最明智的策略——只需把 InterruptedException 传递给方法的调用者。传递 InterruptedException 的方法包括，根本不捕获该异常，或者捕获该异常，然后在执行某种简单的清理工作后再次抛出这个异常。

恢复中断。有时候不能抛出 InterruptedException，例如当代码是 Runnable 的一部分时。在这些情况下，必须捕获 InterruptedException，并通过调用当前线程上的 interrupt 方法恢复中断状态，这样在调用栈中更高层的代码将看到引发了一个中断，如程序清单 5-10 所示。

程序清单 5-10 恢复中断状态以避免屏蔽中断

```
public class TaskRunnable implements Runnable {
    BlockingQueue<Task> queue;
    ...
    public void run() {
        try {
            processTask(queue.take());
        } catch (InterruptedException e) {
            // 恢复被中断的状态
            Thread.currentThread().interrupt();
        }
    }
}
```

还可以采用一些更复杂的中断处理方法，但上述两种方法已经可以应付大多数情况了。然而在出现 InterruptedException 时不应该做的事情是，捕获它但不做出任何响应。这将使调用栈上更高层的代码无法对中断采取处理措施，因为线程被中断的证据已经丢失。只有在一种特殊的情况中才能屏蔽中断，即对 Thread 进行扩展，并且能控制调用栈上所有更高层的代码。第 7 章将进一步介绍取消和中断等操作。

5.5 同步工具类

在容器类中，阻塞队列是一种独特的类：它们不仅能作为保存对象的容器，还能协调生产者和消费者等线程之间的控制流，因为 take 和 put 等方法将阻塞，直到队列达到期望的状态

（队列既非空，也非满）。

同步工具类可以是任何一个对象，只要它根据其自身的状态来协调线程的控制流。阻塞队列可以作为同步工具类，其他类型的同步工具类还包括信号量（Semaphore）、栅栏（Barrier）以及闭锁（Latch）。在平台类库中还包含其他一些同步工具类的类，如果这些类还无法满足需要，那么可以按照第 14 章中给出的机制来创建自己的同步工具类。

所有的同步工具类都包含一些特定的结构化属性：它们封装了一些状态，这些状态将决定执行同步工具类的线程是继续执行还是等待，此外还提供了一些方法对状态进行操作，以及另一些方法用于高效地等待同步工具类进入到预期状态。

5.5.1 闭锁

闭锁是一种同步工具类，可以延迟线程的进度直到其到达终止状态 [CPJ 3.4.2]。闭锁的作用相当于一扇门：在闭锁到达结束状态之前，这扇门一直是关闭的，并且没有任何线程能通过，当到达结束状态时，这扇门会打开并允许所有的线程通过。当闭锁到达结束状态后，将不会再改变状态，因此这扇门将永远保持打开状态。闭锁可以用来确保某些活动直到其他活动都完成后才继续执行，例如：

- 确保某个计算在其需要的所有资源都被初始化之后才继续执行。二元闭锁（包括两个状态）可以用来表示“资源 R 已经被初始化”，而所有需要 R 的操作都必须先在这个闭锁上等待。
- 确保某个服务在其依赖的所有其他服务都已经启动之后才启动。每个服务都有一个相关的二元闭锁。当启动服务 S 时，将首先在 S 依赖的其他服务的闭锁上等待，在所有依赖的服务都启动后会释放闭锁 S，这样其他依赖 S 的服务才能继续执行。
- 等待直到某个操作的所有参与者（例如，在多玩家游戏中的所有玩家）都就绪再继续执行。在这种情况中，当所有玩家都准备就绪时，闭锁将到达结束状态。

CountDownLatch 是一种灵活的闭锁实现，可以在上述各种情况中使用，它可以使一个或多个线程等待一组事件发生。闭锁状态包括一个计数器，该计数器被初始化为一个正数，表示需要等待的事件数量。countDown 方法递减计数器，表示有一个事件已经发生了，而 await 方法等待计数器达到零，这表示所有需要等待的事件都已经发生。如果计数器的值非零，那么 await 会一直阻塞直到计数器为零，或者等待中的线程中断，或者等待超时。

在程序清单 5-11 的 TestHarness 中给出了闭锁的两种常见用法。TestHarness 创建一定数量的线程，利用它们并发地执行指定的任务。它使用两个闭锁，分别表示“起始门（Starting Gate）”和“结束门（Ending Gate）”。起始门计数器的初始值为 1，而结束门计数器的初始值为工作线程的数量。每个工作线程首先要做的值就是在启动门上等待，从而确保所有线程都就绪后才开始执行。而每个线程要做的最后一件事情是将调用结束门的 countDown 方法减 1，这能使主线程高效地等待直到所有工作线程都执行完成，因此可以统计所消耗的时间。

程序清单 5-11 在计时测试中使用 CountDownLatch 来启动和停止线程

```
public class TestHarness {
```

```
    public long timeTasks(int nThreads, final Runnable task)
            throws InterruptedException {
        final CountDownLatch startGate = new CountDownLatch(1);
        final CountDownLatch endGate = new CountDownLatch(nThreads);

        for (int i = 0; i < nThreads; i++) {
            Thread t = new Thread() {
                public void run() {
                    try {
                        startGate.await();
                        try {
                            task.run();
                        } finally {
                            endGate.countDown();
                        }
                    } catch (InterruptedException ignored) { }
                }
            };
            t.start();
        }

        long start = System.nanoTime();
        startGate.countDown();
        endGate.await();
        long end = System.nanoTime();
        return end-start;
    }
}
```

为什么要在 TestHarness 中使用闭锁，而不是在线程创建后就立即启动？或许，我们希望测试 n 个线程并发执行某个任务时需要的时间。如果在创建线程后立即启动它们，那么先启动的线程将“领先”后启动的线程，并且活跃线程数量会随着时间的推移而增加或减少，竞争程度也在不断发生变化。启动门将使得主线程能够同时释放所有工作线程，而结束门则使主线程能够等待最后一个线程执行完成，而不是顺序地等待每个线程执行完成。

5.5.2 FutureTask

FutureTask 也可以用做闭锁。（FutureTask 实现了 Future 语义，表示一种抽象的可生成结果的计算 [CPJ 4.3.3]）。FutureTask 表示的计算是通过 Callable 来实现的，相当于一种可生成结果的 Runnable，并且可以处于以下 3 种状态：等待运行（Waiting to run），正在运行（Running）和运行完成（Completed）。“执行完成”表示计算的所有可能结束方式，包括正常结束、由于取消而结束和由于异常而结束等。当 FutureTask 进入完成状态后，它会永远停止在这个状态上。

Future.get 的行为取决于任务的状态。如果任务已经完成，那么 get 会立即返回结果，否则 get 将阻塞直到任务进入完成状态，然后返回结果或者抛出异常。FutureTask 将计算结果从执行计算的线程传递到获取这个结果的线程，而 FutureTask 的规范确保了这种传递过程能实现结果的安全发布。

FutureTask 在 Executor 框架中表示异步任务，此外还可以用来表示一些时间较长的计算，这些计算可以在使用计算结果之前启动。程序清单 5-12 中的 Preloader 就使用了 FutureTask 来执行一个高开销的计算，并且计算结果将在稍后使用。通过提前启动计算，可以减少在等待结果时需要的时间。

程序清单 5-12　使用 FutureTask 来提前加载稍后需要的数据

```
public class Preloader {
    private final FutureTask<ProductInfo> future =
        new FutureTask<ProductInfo>(new Callable<ProductInfo>() {
            public ProductInfo call() throws DataLoadException {
                return loadProductInfo();
            }
        });
    private final Thread thread = new Thread(future);

    public void start() { thread.start(); }

    public ProductInfo get()
            throws DataLoadException, InterruptedException {
        try {
            return future.get();
        } catch (ExecutionException e) {
            Throwable cause = e.getCause();
            if (cause instanceof DataLoadException)
                throw (DataLoadException) cause;
            else
                throw launderThrowable(cause);
        }
    }
}
```

Preloader 创建了一个 FutureTask，其中包含从数据库加载产品信息的任务，以及一个执行运算的线程。由于在构造函数或静态初始化方法中启动线程并不是一种好方法，因此提供了一个 start 方法来启动线程。当程序随后需要 ProductInfo 时，可以调用 get 方法，如果数据已经加载，那么将返回这些数据，否则将等待加载完成后再返回。

Callable 表示的任务可以抛出受检查的或未受检查的异常，并且任何代码都可能抛出一个 Error。无论任务代码抛出什么异常，都会被封装到一个 ExecutionException 中，并在 Future.get 中被重新抛出。这将使调用 get 的代码变得复杂，因为它不仅需要处理可能出现的 ExecutionException（以及未检查的 CancellationException），而且还由于 ExecutionException 是作为一个 Throwable 类返回的，因此处理起来并不容易。

在 Preloader 中，当 get 方法抛出 ExecutionException 时，可能是以下三种情况之一：Callable 抛出的受检查异常，RuntimeException，以及 Error。我们必须对每种情况进行单独处理，但我们将使用程序清单 5-13 中的 launderThrowable 辅助方法来封装一些复杂的异常处理逻辑。在调用 launderThrowable 之前，Preloader 会首先检查已知的受检查异常，并重新抛出它们。剩下的是未检查异常，Preloader 将调用 launderThrowable 并抛出结果。如果 Throwable 传递给

launderThrowable 的是一个 Error，那么 launderThrowable 将直接再次抛出它；如果不是 RuntimeException，那么将抛出一个 IllegalStateException 表示这是一个逻辑错误。剩下的 RuntimeException，launderThrowable 将把它们返回给调用者，而调用者通常会重新抛出它们。

程序清单 5-13 强制将未检查的 Throwable 转换为 RuntimeException

```
    /** 如果Throwable是Error，那么抛出它；如果是RuntimeException，那么返回它，否则抛出
IllegalStateException。*/
    public static RuntimeException launderThrowable(Throwable t) {
        if (t instanceof RuntimeException)
            return (RuntimeException) t;
        else if (t instanceof Error)
            throw (Error) t;
        else
            throw new IllegalStateException("Not unchecked", t);
    }
```

5.5.3 信号量

计数信号量（Counting Semaphore）用来控制同时访问某个特定资源的操作数量，或者同时执行某个指定操作的数量 [CPJ 3.4.1]。计数信号量还可以用来实现某种资源池，或者对容器施加边界。

Semaphore 中管理着一组虚拟的许可 (permit)，许可的初始数量可通过构造函数来指定。在执行操作时可以首先获得许可（只要还有剩余的许可），并在使用以后释放许可。如果没有许可，那么 acquire 将阻塞直到有许可（或者直到被中断或者操作超时）。release 方法将返回一个许可给信号量。[⊖]计算信号量的一种简化形式是二值信号量，即初始值为 1 的 Semaphore。二值信号量可以用做互斥体 (mutex)，并具备不可重入的加锁语义：谁拥有这个唯一的许可，谁就拥有了互斥锁。

Semaphore 可以用于实现资源池，例如数据库连接池。我们可以构造一个固定长度的资源池，当池为空时，请求资源将会失败，但你真正希望看到的行为是阻塞而不是失败，并且当池非空时解除阻塞。如果将 Semaphore 的计数值初始化为池的大小，并在从池中获取一个资源之前首先调用 acquire 方法获取一个许可，在将资源返回给池之后调用 release 释放许可，那么 acquire 将一直阻塞直到资源池不为空。在第 12 章的有界缓冲类中将使用这项技术。（在构造阻塞对象池时，一种更简单的方法是使用 BlockingQueue 来保存池的资源。）

同样，你也可以使用 Semaphore 将任何一种容器变成有界阻塞容器，如程序清单 5-14 中的 BoundedHashSet 所示。信号量的计数值会初始化为容器容量的最大值。add 操作在向底层容器中添加一个元素之前，首先要获取一个许可。如果 add 操作没有添加任何元素，那么会立刻

⊖ 在这种实现中不包含真正的许可对象，并且 Semaphore 也不会将许可与线程关联起来，因此在一个线程中获得的许可可以在另一个线程中释放。可以将 acquire 操作视为是消费一个许可，而 release 操作是创建一个许可，Semaphore 并不受限于它在创建时的初始许可数量。

释放许可。同样，remove 操作释放一个许可，使更多的元素能够添加到容器中。底层的 Set 实现并不知道关于边界的任何信息，这是由 BoundedHashSet 来处理的。

程序清单 5-14 使用 Semaphore 为容器设置边界

```
public class BoundedHashSet<T> {
    private final Set<T> set;
    private final Semaphore sem;

    public BoundedHashSet(int bound) {
        this.set = Collections.synchronizedSet(new HashSet<T>());
        sem = new Semaphore(bound);
    }

    public boolean add(T o) throws InterruptedException {
        sem.acquire();
        boolean wasAdded = false;
        try {
            wasAdded = set.add(o);
            return wasAdded;
        }
        finally {
            if (!wasAdded)
                sem.release();
        }
    }

    public boolean remove(Object o) {
        boolean wasRemoved = set.remove(o);
        if (wasRemoved)
            sem.release();
        return wasRemoved;
    }
}
```

5.5.4 栅栏

我们已经看到通过闭锁来启动一组相关的操作，或者等待一组相关的操作结束。闭锁是一次性对象，一旦进入终止状态，就不能被重置。

栅栏 (Barrier) 类似于闭锁，它能阻塞一组线程直到某个事件发生 [CPJ 4,4.3]。栅栏与闭锁的关键区别在于，所有线程必须同时到达栅栏位置，才能继续执行。闭锁用于等待事件，而栅栏用于等待其他线程。栅栏用于实现一些协议，例如几个家庭决定在某个地方集合："所有人 6:00 在麦当劳碰头，到了以后要等其他人，之后再讨论下一步要做的事情。"

CyclicBarrier 可以使一定数量的参与方反复地在栅栏位置汇集，它在并行迭代算法中非常有用：这种算法通常将一个问题拆分成一系列相互独立的子问题。当线程到达栅栏位置时将调用 await 方法，这个方法将阻塞直到所有线程都到达栅栏位置。如果所有线程都到达了栅栏位置，那么栅栏将打开，此时所有线程都被释放，而栅栏将被重置以便下次使用。如果对 await 的调用超时，或者 await 阻塞的线程被中断，那么栅栏就被认为是打破了，所有阻塞的 await 调

用都将终止并抛出 BrokenBarrierException。如果成功地通过栅栏，那么 await 将为每个线程返回一个唯一的到达索引号，我们可以利用这些索引来“选举”产生一个领导线程，并在下一次迭代中由该领导线程执行一些特殊的工作。CyclicBarrier 还可以使你将一个栅栏操作传递给构造函数，这是一个 Runnable，当成功通过栅栏时会（在一个子任务线程中）执行它，但在阻塞线程被释放之前是不能执行的。

在模拟程序中通常需要使用栅栏，例如某个步骤中的计算可以并行执行，但必须等到该步骤中的所有计算都执行完毕才能进入下一个步骤。例如，在 n-body 粒子模拟系统中，每个步骤都根据其他粒子的位置和属性来计算各个粒子的新位置。通过在每两次更新之间等待栅栏，能够确保在第 k 步中的所有更新操作都已经计算完毕，才进入第 k+1 步。

在程序清单 5-15 的 CellularAutomata 中给出了如何通过栅栏来计算细胞的自动化模拟，例如 Conway 的生命游戏 (Gardner，1970)。在把模拟过程并行化时，为每个元素（在这个示例中相当于一个细胞）分配一个独立的线程是不现实的，因为这将产生过多的线程，而在协调这些线程上导致的开销将降低计算性能。合理的做法是，将问题分解成一定数量的子问题，为每个子问题分配一个线程来进行求解，之后再将所有的结果合并起来。CellularAutomata 将问题分解为 N_{cpu} 个子问题，其中 N_{cpu} 等于可用 CPU 的数量，并将每个子问题分配给一个线程。[⊖] 在每个步骤中，工作线程都为各自子问题中的所有细胞计算新值。当所有工作线程都到达栅栏时，栅栏会把这些新值提交给数据模型。在栅栏的操作执行完以后，工作线程将开始下一步的计算，包括调用 isDone 方法来判断是否需要进行下一次迭代。

程序清单 5-15 通过 CyclicBarrier 协调细胞自动衍生系统中的计算

```
public class CellularAutomata {
    private final Board mainBoard;
    private final CyclicBarrier barrier;
    private final Worker[] workers;

    public CellularAutomata(Board board) {
        this.mainBoard = board;
        int count = Runtime.getRuntime().availableProcessors();
        this.barrier = new CyclicBarrier(count,
                new Runnable() {
                    public void run() {
                        mainBoard.commitNewValues();
                    }});
        this.workers = new Worker[count];
        for (int i = 0; i < count; i++)
            workers[i] = new Worker(mainBoard.getSubBoard(count, i));
    }

    private class Worker implements Runnable {
        private final Board board;
```

⊖ 在这种不涉及 I/O 操作或共享数据访问的计算问题中，当线程数量为 N_{cpu} 或 $N_{cpu}+1$ 时将获得最优的吞吐量。更多的线程并不会带来任何帮助，甚至在某种程度上会降低性能，因为多个线程将会在 CPU 和内存等资源上发生竞争。

```
        public Worker(Board board) { this.board = board; }
        public void run() {
            while (!board.hasConverged()) {
                for (int x = 0; x < board.getMaxX(); x++)
                    for (int y = 0; y < board.getMaxY(); y++)
                        board.setNewValue(x, y, computeValue(x, y));
                try {
                    barrier.await();
                } catch (InterruptedException ex) {
                    return;
                } catch (BrokenBarrierException ex) {
                    return;
                }
            }
        }
    }

    public void start() {
        for (int i = 0; i < workers.length; i++)
            new Thread(workers[i]).start();
        mainBoard.waitForConvergence();
    }
}
```

另一种形式的栅栏是 Exchanger，它是一种两方（Two-Party）栅栏，各方在栅栏位置上交换数据 [CPJ 3.4.3]。当两方执行不对称的操作时，Exchanger 会非常有用，例如当一个线程向缓冲区写入数据，而另一个线程从缓冲区中读取数据。这些线程可以使用 Exchanger 来汇合，并将满的缓冲区与空的缓冲区交换。当两个线程通过 Exchanger 交换对象时，这种交换就把这两个对象安全地发布给另一方。

数据交换的时机取决于应用程序的响应需求。最简单的方案是，当缓冲区被填满时，由填充任务进行交换，当缓冲区为空时，由清空任务进行交换。这样会把需要交换的次数降至最低，但如果新数据的到达率不可预测，那么一些数据的处理过程就将延迟。另一个方法是，不仅当缓冲被填满时进行交换，并且当缓冲被填充到一定程度并保持一定时间后，也进行交换。

5.6 构建高效且可伸缩的结果缓存

几乎所有的服务器应用程序都会使用某种形式的缓存。重用之前的计算结果能降低延迟，提高吞吐量，但却需要消耗更多的内存。

像许多“重复发明的轮子”一样，缓存看上去都非常简单。然而，简单的缓存可能会将性能瓶颈转变成可伸缩性瓶颈，即使缓存是用于提升单线程的性能。本节我们将开发一个高效且可伸缩的缓存，用于改进一个高计算开销的函数。我们首先从简单的 HashMap 开始，然后分析它的并发性缺陷，并讨论如何修复它们。

在程序清单 5-16 的 Computable<A，V> 接口中声明了一个函数 Computable，其输入类型为 A，输出类型为 V。在 ExpensiveFunction 中实现的 Computable，需要很长的时间来计算结果，我们将创建一个 Computable 包装器，帮助记住之前的计算结果，并将缓存过程封装起来。（这项技术被称为“记忆 [Memoization]”。）

程序清单 5-16 使用 HashMap 和同步机制来初始化缓存

```
public interface Computable<A, V> {
    V compute(A arg) throws InterruptedException;
}

public class ExpensiveFunction
        implements Computable<String, BigInteger> {
    public BigInteger compute(String arg) {
        // 在经过长时间的计算后
        return new BigInteger(arg);
    }
}

public class Memoizer1<A, V> implements Computable<A, V> {
    @GuardedBy("this")
    private final Map<A, V> cache = new HashMap<A, V>();
    private final Computable<A, V> c;

    public Memoizer1(Computable<A, V> c) {
        this.c = c;
    }

    public synchronized V compute(A arg) throws InterruptedException {
        V result = cache.get(arg);
        if (result == null) {
            result = c.compute(arg);
            cache.put(arg, result);
        }
        return result;
    }
}
```

在程序清单 5-16 中的 Memoizer1 给出了第一种尝试：使用 HashMap 来保存之前计算的结果。compute 方法将首先检查需要的结果是否已经在缓存中，如果存在则返回之前计算的值。否则，将把计算结果缓存在 HashMap 中，然后再返回。

HashMap 不是线程安全的，因此要确保两个线程不会同时访问 HashMap，Memoizer1 采用了一种保守的方法，即对整个 compute 方法进行同步。这种方法能确保线程安全性，但会带来一个明显的可伸缩性问题：每次只有一个线程能够执行 compute。如果另一个线程正在计算结果，那么其他调用 compute 的线程可能被阻塞很长时间。如果有多个线程在排队等待还未计算出的结果，那么 compute 方法的计算时间可能比没有“记忆”操作的计算时间更长。在图 5-2 中给出了当多个线程使用这种方法中的“记忆”操作时发生的情况，这显然不是我们希望通过缓存获得的性能提升结果。

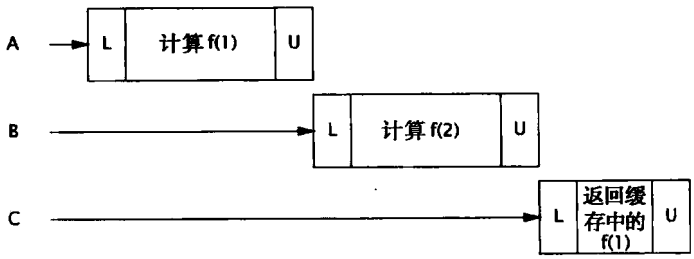

图 5-2　Memoizer1 糟糕的并发性

程序清单 5-17 中的 Memoizer2 用 ConcurrentHashMap 代替 HashMap 来改进 Memoizer1 中糟糕的并发行为。由于 ConcurrentHashMap 是线程安全的，因此在访问底层 Map 时就不需要进行同步，因而避免了在对 Memoizer1 中的 compute 方法进行同步时带来的串行性。

Memoizer2 比 Memoizer1 有着更好的并发行为：多线程可以并发地使用它。但它在作为缓存时仍然存在一些不足——当两个线程同时调用 compute 时存在一个漏洞，可能会导致计算得到相同的值。在使用 memoization 的情况下，这只会带来低效，因为缓存的作用是避免相同的数据被计算多次。但对于更通用的缓存机制来说，这种情况将更为糟糕。对于只提供单次初始化的对象缓存来说，这个漏洞就会带来安全风险。

程序清单 5-17　用 ConcurrentHashMap 替换 HashMap

```
public class Memoizer2<A, V> implements Computable<A, V> {
    private final Map<A, V> cache = new ConcurrentHashMap<A, V>();
    private final Computable<A, V> c;

    public Memoizer2(Computable<A, V> c) { this.c = c; }

    public V compute(A arg) throws InterruptedException {
        V result = cache.get(arg);
        if (result == null) {
            result = c.compute(arg);
            cache.put(arg, result);
        }
        return result;
    }
}
```

Memoizer2 的问题在于，如果某个线程启动了一个开销很大的计算，而其他线程并不知道这个计算正在进行，那么很可能会重复这个计算，如图 5-3 所示。我们希望通过某种方法来表达“线程 X 正在计算 f(27)”这种情况，这样当另一个线程查找 f(27) 时，它能够知道最高效的方法是等待线程 X 计算结束，然后再去查询缓存“f(27) 的结果是多少？”。

我们已经知道有一个类能基本实现这个功能：FutureTask。FutureTask 表示一个计算的过程，这个过程可能已经计算完成，也可能正在进行。如果有结果可用，那么 FutureTask.get 将立即返回结果，否则它会一直阻塞，直到结果计算出来再将其返回。

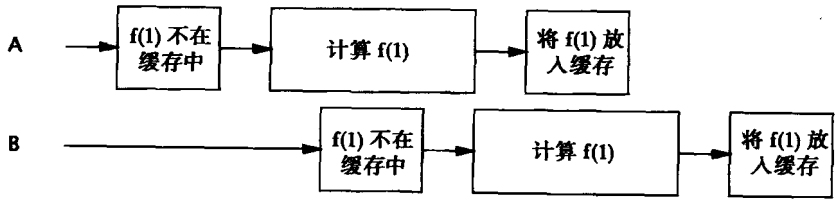

图 5-3 当使用 Memoizer2 时，两个线程计算相同的值

程序清单 5-18 中的 Memoizer3 将用于缓存值的 Map 重新定义为 ConcurrentHashMap<A, Future<V>>，替换原来的 ConcurrentHashMap<A, V>。Memoizer3 首先检查某个相应的计算是否已经开始（Memoizer2 与之相反，它首先判断某个计算是否已经完成）。如果还没有启动，那么就创建一个 FutureTask，并注册到 Map 中，然后启动计算：如果已经启动，那么等待现有计算的结果。结果可能很快会得到，也可能还在运算过程中，但这对于 Future.get 的调用者来说是透明的。

程序清单 5-18 基于 FutureTask 的 Memoizing 封装器

```
public class Memoizer3<A, V> implements Computable<A, V> {
    private final Map<A, Future<V>> cache
            = new ConcurrentHashMap<A, Future<V>>();
    private final Computable<A, V> c;

    public Memoizer3(Computable<A, V> c) { this.c = c; }

    public V compute(final A arg) throws InterruptedException {
        Future<V> f = cache.get(arg);
        if (f == null) {
            Callable<V> eval = new Callable<V>() {
                public V call() throws InterruptedException {
                    return c.compute(arg);
                }
            };
            FutureTask<V> ft = new FutureTask<V>(eval);
            f = ft;
            cache.put(arg, ft);
            ft.run(); // 在这里将调用 c.compute
        }
        try {
            return f.get();
        } catch (ExecutionException e) {
            throw launderThrowable(e.getCause());
        }
    }
}
```

Memoizer3 的实现几乎是完美的：它表现出了非常好的并发性（基本上是源于 ConcurrentHashMap 高效的并发性），若结果已经计算出来，那么将立即返回。如果其他线程正在计算该结果，那么新到的线程将一直等待这个结果被计算出来。它只有一个缺陷，即仍然存在两个线程计算出相同值的漏洞。这个漏洞的发生概率要远小于 Memoizer2 中发生的概率，但

由于 compute 方法中的 if 代码块仍然是非原子 (nonatomic) 的“先检查再执行”操作，因此两个线程仍有可能在同一时间内调用 compute 来计算相同的值，即二者都没有在缓存中找到期望的值，因此都开始计算。这个错误的执行时序如图 5-4 所示。

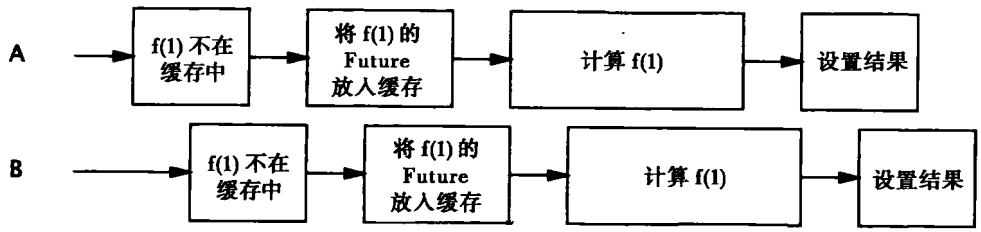

图 5-4 错误的执行时序将使得 Memorizer3 将相同的值计算两次

Memoizer3 中存在这个问题的原因是，复合操作（“若没有则添加”）是在底层的 Map 对象上执行的，而这个对象无法通过加锁来确保原子性。程序清单 5-19 中的 Memoizer 使用了 ConcurrentMap 中的原子方法 putIfAbsent，避免了 Memoizer3 的漏洞。

程序清单 5-19 Memoizer 的最终实现

```
public class Memoizer<A, V> implements Computable<A, V> {
    private final ConcurrentMap<A, Future<V>> cache
        = new ConcurrentHashMap<A, Future<V>>();
    private final Computable<A, V> c;

    public Memoizer(Computable<A, V> c) { this.c = c; }

    public V compute(final A arg) throws InterruptedException {
        while (true) {
            Future<V> f = cache.get(arg);
            if (f == null) {
                Callable<V> eval = new Callable<V>() {
                    public V call() throws InterruptedException {
                        return c.compute(arg);
                    }
                };
                FutureTask<V> ft = new FutureTask<V>(eval);
                f = cache.putIfAbsent(arg, ft);
                if (f == null) { f = ft; ft.run(); }
            }
            try {
                return f.get();
            } catch (CancellationException e) {
                cache.remove(arg, f);
            } catch (ExecutionException e) {
                throw launderThrowable(e.getCause());
            }
        }
    }
}
```

当缓存的是 Future 而不是值时，将导致缓存污染（Cache Pollution）问题：如果某个

计算被取消或者失败，那么在计算这个结果时将指明计算过程被取消或者失败。为了避免这种情况，如果 Memoizer 发现计算被取消，那么将把 Future 从缓存中移除。如果检测到 RuntimeException, 那么也会移除 Future，这样将来的计算才可能成功。Memoizer 同样没有解决缓存逾期的问题，但它可以通过使用 FutureTask 的子类来解决，在子类中为每个结果指定一个逾期时间，并定期扫描缓存中逾期的元素。（同样，它也没有解决缓存清理的问题，即移除旧的计算结果以便为新的计算结果腾出空间，从而使缓存不会消耗过多的内存。）

在完成并发缓存的实现后，就可以为第 2 章中因式分解 servlet 添加结果缓存。程序清单 5-20 中的 Factorizer 使用 Memoizer 来缓存之前的计算结果，这种方式不仅高效，而且可扩展性也更高。

程序清单 5-20 在因式分解 servlet 中使用 Memoizer 来缓存结果

```
@ThreadSafe
public class Factorizer implements Servlet {
    private final Computable<BigInteger, BigInteger[]> c =
        new Computable<BigInteger, BigInteger[]>() {
            public BigInteger[] compute(BigInteger arg) {
                return factor(arg);
            }
        };
    private final Computable<BigInteger, BigInteger[]> cache
        = new Memoizer<BigInteger, BigInteger[]>(c);

    public void service(ServletRequest req,
                        ServletResponse resp) {
        try {
            BigInteger i = extractFromRequest(req);
            encodeIntoResponse(resp, cache.compute(i));
        } catch (InterruptedException e) {
            encodeError(resp, "factorization interrupted");
        }
    }
}
```

第一部分小结

到目前为止，我们已经介绍了许多基础知识。下面这个“并发技巧清单”列举了在第一部分中介绍的主要概念和规则。

- 可变状态是至关重要的（It's the mutable state, stupid）[⊖]。

  所有的并发问题都可以归结为如何协调对并发状态的访问。可变状态越少，就越容易确保线程安全性。

⊖ 在 1992 年美国总统竞选期间，竞选策略专家 James Carville 在 Bill Clinton 的竞选总部里挂了一个牌子，上面写着“The economy, stupid（经济是至关重要的）”，作为竞选运动的核心理念。

- 尽量将域声明为 final 类型，除非需要它们是可变的。
- 不可变对象一定是线程安全的。

  不可变对象能极大地降低并发编程的复杂性。它们更为简单而且安全，可以任意共享而无须使用加锁或保护性复制等机制。
- 封装有助于管理复杂性。

  在编写线程安全的程序时，虽然可以将所有数据都保存在全局变量中，但为什么要这样做？将数据封装在对象中，更易于维持不变性条件：将同步机制封装在对象中，更易于遵循同步策略。
- 用锁来保护每个可变变量。
- 当保护同一个不变性条件中的所有变量时，要使用同一个锁。
- 在执行复合操作期间，要持有锁。
- 如果从多个线程中访问同一个可变变量时没有同步机制，那么程序会出现问题。
- 不要故作聪明地推断出不需要使用同步。
- 在设计过程中考虑线程安全，或者在文档中明确地指出它不是线程安全的。
- 将同步策略文档化。